

Uczenie Maszynowe Lista 8

Mateusz Olech

22 czerwca 2026

1 EDA

Link do notatnika z kaggle: Obesity Risk Prediction (Majority Voting with OOF)

(gdyby nie działał: <https://www.kaggle.com/code/neupane9sujal/obesity-risk-prediction-majority-voting-with-oof>)

Mój notatnik: Lista 5 - obesity

Przeważnie prace konkursowe ograniczają się wyłącznie do wizualizacji jak wyglądają dataframe'y oraz ewentualnie jaki jest rozkład klasy docelowej. Rzadko na tym etapie (albo po prostu nie załączają tego w ostatecznych wersjach) skupiają się na korelacjach między danymi czy charakterystyką cech liczbowych.

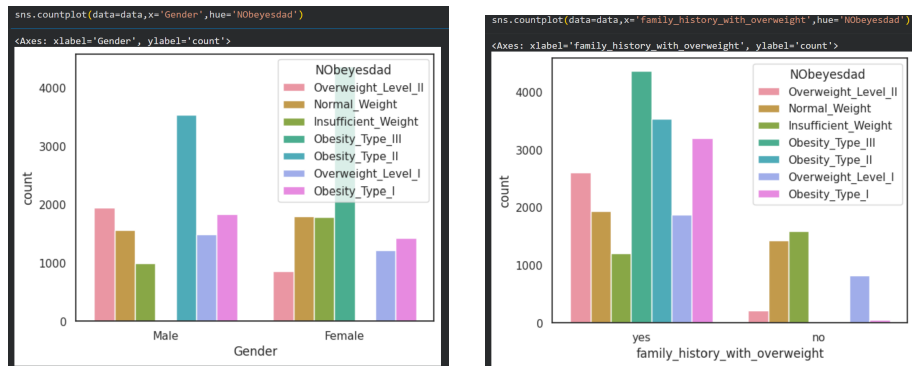
Rozkład klas:

```
df['NObeyesdad'].value_counts().sort_index().plot(
    kind='bar',
    color='steelblue'
)

plt.title('Rozkład klas (0 - Niedowaga, 6 - Otyłość III)')
plt.show() # mój wykres
```

```
sns.countplot(data=data, x='NObeyesdad')
plt.xticks(rotation=75)
plt.show() # jego wykres
```

Ja zastosowałem macierz korelacji, a autor projektu przedstawił wizualizacje cech o prawdopodobnej korelacji (niewiadomo jak do tego doszedł)



2 Preprocessing

Oboje mapujemy cechy tekstowe na numeryczne, aby miały zastosowanie w modelach. Różnica tkwi w tym, że autor pracy konkursowej robi to w ramach klasy Preprocessing.

```
# MÓJ FRAGMENT
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df['family_history_with_overweight'] = df['family_history_with_overweight'].map({'yes': 1, 'no': 0})
df['FAVC'] = df['FAVC'].map({'yes': 1, 'no': 0})
df['SMOKE'] = df['SMOKE'].map({'yes': 1, 'no': 0})
df['SCC'] = df['SCC'].map({'yes': 1, 'no': 0})

df['CAEC'] = df['CAEC'].map({
    'no': 0,
    'Sometimes': 1,
    'Frequently': 2,
    'Always': 3
})

df['CALC'] = df['CALC'].map({
    'no': 0,
    'Sometimes': 1,
    'Frequently': 2,
    'Always': 3
})

df['MTRANS'] = df['MTRANS'].map({
    'Walking': 0,
    'Public_Transportation': 1,
    'Automobile': 2,
    'Motorbike': 3,
    'Bike': 4
})
```

```

df['Passive_Transport'] = df['MTRANS'].map({
    0: 0,
    4: 0,
    1: 1,
    2: 1,
    3: 1
})

df.head()

```

```

# KOD KONKURSOWY
class Preprocessor:
    def __init__(self, df, test, original):
        self.data = df
        self.test = test
        self.original_data = original

    def encode(self, data):
        transformed_data = data.copy()

        transformed_data['Gender'] = (
            transformed_data['Gender']
            .map({"Male": 1, "Female": 0})
            .astype(np.uint8)
        )

        transformed_data['family_history_with_overweight'] = (
            transformed_data.family_history_with_overweight
            .map({"yes": 1, "no": 0})
            .astype(np.uint8)
        )

        transformed_data['FAVC'] = (
            transformed_data.FAVC
            .map({"yes": 1, "no": 0})
            .astype(np.uint8)
        )

        transformed_data['CAEC'] = (
            transformed_data.CAEC
            .map({"Sometimes": 0,
                  "Frequently": 1,
                  "no": 2,
                  "Always": 3})
            .astype(np.uint8)
        )

        transformed_data.SMOKE = (

```

```

        transformed_data.SMOKE
        .map({"yes": 1, "no": 0})
        .astype(np.uint8)
    )

    transformed_data.SCC = (
        transformed_data.SCC
        .map({"yes": 1, "no": 0})
        .astype(np.uint8)
    )

    transformed_data.CALC = (
        transformed_data.CALC
        .map({"Sometimes": 0,
              "Frequently": 1,
              "no": 2})
    )

    transformed_data.MTRANS = (
        transformed_data.MTRANS
        .map({"Public_Transportation": 0,
              "Automobile": 1,
              "Walking": 2,
              "Motorbike": 3,
              "Bike": 4})
        .astype(np.uint8)
    )

    return transformed_data

def transform_data(self, mode):
    if mode == 'train':
        transformed_data = self.encode(self.data)
    else:
        transformed_data = self.encode(self.test)

    return transformed_data

```

Dodatkowo na podstawie wniosków z macierzy korelacji, postanowiłem usunąć nie mające większego znaczenia cechy (jak np. SMOKE czy MTRANS) natomiast w drugiej pracy autor usunął tylko wskaźnik ID. W ramach własnej inicjatywy dodałem również nową cechę bazującą na środkach transportu.

Oboje oddzielamy klasę docelową od pozostałych cech.

Kolejna różnica to sposób, w jaki dzielimy dane na zbiory treningowy i testowy. Wynika ona przede wszystkim z faktu, że w pracy konkursowej autor skorzystał z dedykowanego, z góry narzuconego zbioru testowego, podczas gdy ja dysponowałem tylko jednym plikiem z danymi. W moim przypadku konieczne było sztuczne wydzielenie izolowanego zbioru testowego (za pomocą `train_test_split`), aby móc wiarygodnie ocenić zdolność modelu do generalizacji na nowych pacjen-

tach. Z kolei autor na platformie Kaggle, mając już oddzielny zbiór testowy (bez etykiet), nie musiał 'odcinać' części danych. Zamiast tego zastosował walidację krzyżową (technikę Out-of-Fold), co pozwoliło mu na wykorzystanie 100% oetykietowanych danych do procesu uczenia i wyciśnięcie maksymalnej skuteczności z modelu.

```
# MÓJ PODZIAŁ
X = df.drop('NObeyesdad', axis=1)
y = df['NObeyesdad']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(f"Rozmiar zbioru treningowego: {X_train.shape[0]}")
print(f"Rozmiar zbioru testowego: {X_test.shape[0]}")
```

3 Optymalizacja hiperparametrów

W tym podpunkcie dochodzi do pierwszej znaczącej różnicy między naszymi pracami. Ja do optymalizacji hiperparametrów zastosowałem GridSearchCV, za pomocą którego znajdowałem najlepszą kombinację hiperparametrów spośród podanej siatki. Przykładowy fragment z mojego notatnika:

```
param_grid = {
    'max_depth': [3, 4, 5, 6, 7, 8],
    'criterion': ['gini', 'entropy'],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='recall',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_
```

Z kolei notatnik z Kaggle, ze względu na wykorzystanie modeli gradientowych (XGB/LGBM) o dużej złożoności parametrycznej, zrezygnował z przeszukiwania siatki na rzecz optymalizacji bayesowskiej (biblioteka optuna). W tym po-

dejsi u definiowana jest funkcja objective, a algorytm w sposób probabilistyczny porusza się po wielowymiarowej przestrzeni parametrów (np. learning_rate, n_estimators). Pokazuje to, że w przypadku potężnych komitetów (Ensembles), optymalizacja probabilistyczna staje się jedyną użyteczną metodą radzenia sobie z eksplozją kombinatoryczną.

```
def objective(self, trial):
    target_col = ['NObeyesdad']

    train_cols = self.num_cols + self.cat_cols
    test_predict_list = []
    acc_score_list = []
    all_test_predictions = []

    for i in range(1):
        rkf = RepeatedKFold(
            n_splits=5,
            n_repeats=1,
            random_state=42 + i * i
        )

        for fold, (train_idx, valid_idx) in enumerate(
            rkf.split(self.train[train_cols], self.train[
                target_col])
        ):
            X_train = self.train.iloc[train_idx][train_cols]
            y_train = np.array(self.train.iloc[train_idx][
                target_col]).ravel()

            X_valid = self.train.iloc[valid_idx][train_cols]
            y_valid = np.array(self.train.iloc[valid_idx][
                target_col]).ravel()

            model = LGBMClassifier(
                objective="multiclass",
                metric="multi_logloss",
                verbosity=-1,
                boosting_type="gbdt",
                random_state=0,
                num_class=7,
                learning_rate=trial.suggest_uniform('
                    learning_rate', 0.01, 0.3),
                n_estimators=trial.suggest_int('n_estimators
                    ', 100, 1000),
                lambda_l1=trial.suggest_uniform('lambda_l1',
                    0.0, 1.0),
                lambda_l2=trial.suggest_uniform('lambda_l2',
                    0.0, 1.0),
                feature_fraction=trial.suggest_uniform('
                    feature_fraction', 0.25, 1.0),
```

```

        max_depth=trial.suggest_int('max_depth', 5,
                                     20),
        colsample_bytree=trial.suggest_uniform('
            colsample_bytree', 0.4, 1.0),
        subsample=trial.suggest_uniform('subsample',
                                         0.5, 1.0),
        min_child_samples=trial.suggest_int('
            min_child_samples', 5, 50),
        device="gpu"
    )

    early_stopping_callback = early_stopping(
        100,
        first_metric_only=True,
        verbose=False
    )
    verbose_callback = log_evaluation(100)

    model.fit(
        X_train,
        y_train,
        eval_set=[(X_valid, y_valid)],
        callbacks=[early_stopping_callback,
                  verbose_callback],
        eval_metric='multi_logloss'
    )

    valid_preds = model.predict(X_valid)
    test_preds = model.predict(self.test[train_cols
    ])

    test_predict_list.append(test_preds)

    acc_score = accuracy_score(y_valid, valid_preds)
    acc_score_list.append(acc_score)

    return np.mean(np.array(acc_score_list))

def find_params(self):
    study = optuna.create_study(direction='maximize')
    study.optimize(self.objective, 350)

    best_params = study.best_params
    return best_params

```

4 Uczenie modeli

Kolejny podpunkt, coraz więcej różnic. Do trenowania swoich modeli zastosował metodę `.fit(X_train, y_train)`, przez co model pracował na pełnym zbiorze treningowym oraz skorzystałem z walidacji krzyżowej do uniknięcia przeuczenia modelu.

```
cv_strategy = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cv_scores = cross_val_score(
    tree,
    X_train,
    y_train,
    cv=cv_strategy,
    scoring="f1_macro"
)

tree.fit(X_train, y_train)
```

W przypadku pracy konkursowej autor nie trenuje jednego, wielkiego modelu LightGBM na wszystkich danych. Zamiast tego tworzy pętlę (np. 5-krotną). W każdej iteracji model uczy się na 4 częściach danych, a 5 część służy mu do dynamicznej oceny.

Dodatkowo korzysta ze wsparcia `*early_stopping`, który gwarantuje wcześniejsze zatrzymanie treningu w przypadku, gdy model jest zagrożony overfittingiem. Model uczy się na 4 częściach, po każdym dodanym drzewie sprawdzając swój błąd na 5 części. Gdy błąd rośnie, trening jest natychmiast przerywany.

```
def train_lgb(self):
    target_col = ['NObeyesdad']

    train_cols = self.num_cols + self.cat_cols
    test_predict_list = []
    all_test_predictions = []

    scores = list()

    for i in range(1):
        print(f"Iteration:{i+1}")

        rkf = RepeatedKFold(
            n_splits=30,
            n_repeats=1,
            random_state=42 + i * i
        )
```

```

for fold, (train_idx, valid_idx) in enumerate(
    rkf.split(self.train[train_cols], self.train[
        target_col]))
):
    X_train = self.train.iloc[train_idx][train_cols]
    y_train = np.array(self.train.iloc[train_idx][
        target_col]).ravel()

    X_valid = self.train.iloc[valid_idx][train_cols]
    y_valid = np.array(self.train.iloc[valid_idx][
        target_col]).ravel()

    model = LGBMClassifier(
        objective="multiclass",
        metric="multi_logloss",
        verbosity=-1,
        random_state=0,
        learning_rate=0.05828871870812271,
        n_estimators=159,
        lambda_l1=0.15395856554089996,
        lambda_l2=0.2913233407307633,
        feature_fraction=0.4193073138371244,
        max_depth=20,
        colsample_bytree=0.918042564259133,
        subsample=0.9602243741487563,
        min_child_samples=47,
        device="gpu"
    )

    early_stopping_callback = early_stopping(
        100,
        first_metric_only=True,
        verbose=False
    )
    verbose_callback = log_evaluation(100)

    model.fit(
        X_train,
        y_train,
        eval_set=[(X_valid, y_valid)],
        callbacks=[early_stopping_callback,
            verbose_callback],
        eval_metric='multi_logloss'
    )

    valid_preds = model.predict(X_valid)
    test_preds = model.predict(self.test[train_cols
    ])

    test_predict_list.append(test_preds)

```

```

        acc_score = accuracy_score(y_valid, valid_preds)
        scores.append(acc_score)

        print(f"Fold: {fold+1}\n==>Accuracy score: {
            acc_score}")

        all_test_predictions.extend(test_predict_list)

    print(f"The average accuracy score is {np.mean(np.array(
        scores))}")

    predictions_3d = np.array(all_test_predictions)

    final_predictions = []
    for entry_predictions in predictions_3d.T:
        unique, counts = np.unique(entry_predictions,
            return_counts=True)
        final_predictions.append(unique[np.argmax(counts)])

    return final_predictions

```

5 Ewaluacja modeli

Różnica między projektami jest znacząca, ale ona wynika przede wszystkim z głównych założeń początkowych dla każdego z notatników. Praca konkursowa miała skupić się na maksymalizacji dokładności, przez co autor wybrał tę metrykę jako docelowy wyznacznik modelu.

```

        acc_score = accuracy_score(y_valid, valid_preds)
        acc_score_list.append(acc_score)

    return np.mean(np.array(acc_score_list))

def find_params(self):
    study = optuna.create_study(direction='maximize')
    study.optimize(self.objective, 350)

    best_params = study.best_params
    return best_params

```

Z kolei w mojej pracy analizuję różne metryki i staram się znaleźć jak najlepszy kompromis między metrykami precyzji i czułości (z trochę większym naciskiem na czułość). Wymusza to jednak mocniejsze skupienie na ogólnym raporcie z klasyfikacji niż na maksymalizacji jednej miary. Wpływ na ocenę modeli ma również założenie o warunkach technicznych w meksykańskiej służbie zdrowia.

```

print(f"Czas predykcji: {inference_time:.4f}s")

target_names = [
    'Niedowaga (0)', 'Waga normalna (1)', 'Nadwaga I (2)',
    'Nadwaga II (3)', 'Otyłość I (4)', 'Otyłość II (5)', '
    Otyłość III (6)'
]

print(classification_report(y_test, y_pred, target_names=
    target_names))

```

Klasa	Precision	Recall	F1-score	Support
Niedowaga (0)	0.80	0.83	0.82	54
Waga normalna (1)	0.49	0.48	0.49	58
Nadwaga I (2)	0.62	0.69	0.65	58
Nadwaga II (3)	0.72	0.66	0.68	58
Otyłość I (4)	0.78	0.74	0.76	70
Otyłość II (5)	0.90	0.92	0.91	60
Otyłość III (6)	0.98	0.97	0.98	65
Accuracy		0.76		423
Macro avg	0.76	0.76	0.76	423
Weighted avg	0.76	0.76	0.76	423